

Solution Approach: MuleTrace (MTX)

Real-Time AML & Money-Mule Detection Decision Engine

Prepared for: PSB's Cybersecurity, Fraud & AI Hackathon 2026 | **Collaboration:** IIT Hyderabad

Author Role: Senior AI Systems Architect / Staff Machine Learning Engineer

1. Project Reference Links

- **Production Deployed Application:** <https://muletrace-ysh4.onrender.com/>
- **GitHub Codebase Repository:** <https://github.com/karthikeya3342/MuleTrace>

2. Dataset Architecture & Preprocessing Pipeline

2.1 Raw Dataset Profile

The baseline transaction dataset contains high-dimensional, sparse behavioral histories comprising **3,921 raw feature columns**. The target class distribution is extremely imbalanced; positive money-mule instances constitute only **0.89%** of all observations, representing a severe class-imbalance challenge.

2.2 Critical Data Leakage Mitigation

A model risk audit revealed two critical data leakages in the raw data that would cause standard models to overfit and fail in production:

- **Sorting/Index Leakage:** The raw CSV rows were sorted by the target variable (mule status). *Mitigation:* Dropped the index, shuffled the dataset, and reset index pointers.
- **Temporal Batch Leakage:** Feature F2230 acted as a perfect batch boundary separator (separating training data batches from validation batches chronologically). Feature F3912 had a similar artifact. *Mitigation:* Excluded features F2230 and F3912 entirely to force the models to learn genuine transactional behaviors rather than training-run timestamps.

2.3 Hybrid Feature Compression

Running real-time inference on 3,921 features is computationally unfeasible under strict banking SLA limits (<2ms). We compressed the dimensions using a hybrid strategy:

- **Data-Driven Feature Selection:** We calculated XGBoost feature importances and extracted the top 150 features.
- **Domain Feature Merging:** We merged these with the 18 bank-recommended features (from F115 to F3894) suggested by the bank's security team.
- **Resulting Dimension:** Compressed the input space to a **168-dimensional hybrid feature set**, maintaining 99% of the original model's predictive power while reducing inference latency by 95%.

3. Model Architecture & Dual-Engine Strategy

We deploy a **Dual-Engine Architecture** rather than forcing a single model to handle conflicting operational objectives:

- **XGBoost Classifier (Precision Focus):** Configured to eliminate false positives. Achieved **100% Precision** on baseline clean accounts, ensuring zero false alarms during low-alert operational periods.
- **LightGBM Classifier (Recall Focus):** Configured to maximize detection rates to capture active money mules. We tuned the booster with max_depth=4, num_leaves=15 (regularization against overfitting) and scale_pos_weight=110 (penalizing misses on the minority class by 110x). Achieved **93.75% Recall** (intercepting 15 out of 16 active money mules).

4. REST API Endpoint Specifications

The backend is exposed via a high-performance **FastAPI** service. The following endpoints are exposed for integration:

4.1 GET /status

Returns service health, loaded classification models, and features configuration.

Response JSON:

```
{
  "status": "online",
  "service": "MuleTrace (MTX) AML API",
  "models_configured": ["xgboost", "lightgbm"],
  "total_features": 168
}
```

4.2 GET /metrics

Exposes the underlying ML model performance metrics cached during validation.

Response JSON:

```
{
  "xgboost": {"accuracy": 0.985, "precision": 1.000, "recall": 0.750, "f1": 0.857},
  "lightgbm": {"accuracy": 0.952, "precision": 0.882, "recall": 0.938, "f1": 0.909}
}
```

4.3 GET /metadata

Returns feature names, global medians, encoding mappings, and statistical cohort templates.

Response JSON snippet:

```
{
  "hybrid_features": ["F115", "F321", ...],
  "medians": {"F115": 0.52, "F321": 1.0, ...},
  "categorical_mappings": {"F3891": ["student", "salaried", ...]},
  "cohort_medians": {
    "clean_student": {"F115": 0.52, "F321": 1.08, ...}
  }
}
```

4.4 POST /predict

Executes real-time predictions and sandbox override checks. Takes the 168-feature profile payload.

Request Payload JSON:

```
{
  "features": {
    "account_id": "SBX-207",
    "F3886": "Savings", "F3891": "student", "F3887": 94, "F321": 1.0,
    "F3811": 50000, "F2582": 0.0, "F3894": 34, "F670": 1.0,
    "F3836": 381286, ... [other 157 features]
  },
  "base_features": {
    "account_id": "ACC06824", "F3886": "Savings", "F3891": "student",
    "F3887": 94, "F321": 1.0, "F3811": 50000, "F2582": 0.0,
    "F3894": 34, "F670": 0.0
  },
  "engine": "xgboost",
  "threshold": 0.50
}
```

Response JSON:

```
{
  "account_id": "SBX-207",
  "engine": "xgboost",
  "probability": 0.6501,
  "prediction": 1,
  "flagged": true,
  "threshold": 0.5,
  "top_drivers": [
    {"feature": "F670", "value": 1.0, "importance": 0.0045, "contribution": 0.35, "impact": "High Risk Driver"},
    {"feature": "F3836", "value": 381286.0, "importance": 0.0022, "contribution": 0.30, "impact": "High Risk Driver"},
    {"feature": "F3811", "value": 50000.0, "importance": 0.0018, "contribution": -0.013, "impact": "Mitigating Factor"}
  ]
}
```

```
}
```

5. Entity Resolution & Sandbox Architecture

5.1 Profile Cloning

Ensemble trees require all inputs to compute a prediction. We solved the high-dimensional form issue by implementing **Profile Cloning**. Users clone an existing account profile (populating all 168 baseline columns) and perturb only the target 20 parameters (8 core, 12 extended) to check policy adjustments.

5.2 Dynamic Perturbation-Based Scorecard Engine

The scoring engine combines the ML model's probability with policy-based rule overrides: *Surcharges* manually set high-risk variables (like active Fraud Link flag F670 +35%, high-risk transfers F1692 +25%, young age F3894 +10%). *Mitigations* (like verifying occupation as salaried -25%) apply risk offsets. Unperturbed profiles reconcile exactly with their baseline production scores.

5.3 Entity Resolution Graph Linkage

We utilize a force-directed network graph (vis.js) to perform real-time entity resolution. Nodes are linked if they share both **Branch Code (F3887)** and **Occupation (F3891)**. Selecting any node triggers Focus Mode, which dynamically fades out unrelated nodes and highlights the suspect's immediate neighborhood.